CollabIQ

AI-Powered Project Team Formation Platform

Software Engineering Documentation

Combined Team Report — Chapters 1–5

Member 1 — Functional Requirements & User Stories
Member 2 — Non-Functional Requirements, Scope & Tech Stack
Member 3 — Use Case Diagram & User Flow
Member 4 — System Architecture & API Planning
Member 5 — Database Design (ERD)

CollabIQ Platform Documentation — June 2026

CHAPTER 1

Functional Requirements, User Stories & System Actors

Member 1 — Requirements Lead

1. Functional Requirements, User Stories & System Actors

This chapter defines the functional requirments ,user stories and system actors of the CollabIQ platform . It describes the main system functionalities , identifies the users and external systems that interact with the platform and outlines the users the user goals that guide the system design. These requirments provide the foundation for the use cases ,architecture, APIs , and database design presented in the following chapters.

1.1 System Actors

Three actors interact with the CollabIQ system. These are the same actors used consistently across the use case diagram (Chapter 3) and the API design (Chapter 4).

Actor	Type	Description
Student	Primary, human	The end user of the platform. Registers, builds a profile, and either creates a team (becoming its Team Leader) or is invited into one. As Team Leader, a student can send invitations, request AI-suggested candidates, and manage the team's project and tasks. Any student can create tasks, accept or decline invitations, and track their own reputation.
AI System	Secondary, system	The backend AI service responsible for three distinct functions: suggesting candidate teammates (Team Matching), suggesting candidate projects (Project Recommendation), and generating a phased roadmap once a project is selected. The AI System never modifies team membership, never creates a project on its own, and is always invoked through the Application Layer on a student's behalf — it has no direct interface with the user.
Admin	Secondary, human	Maintains the platform's shared reference data: the global skills catalog, the interests catalog, and the role catalog used throughout invitations, matching, and role proposals. The Admin does not participate in any individual team's workflow.

1.2 Functional Requirements

Functional requirements are grouped into the same four functional areas used in the use case diagram: Management, Team Formation, Project Planning, and Project Management.

1.2.1 Management (Account & Profile)

- FR-1.1 — The system shall allow a student to register a new account with full name, email, and password.
- FR-1.2 — The system shall allow a registered student to log in using email and password and receive a JWT access token.
- FR-1.3 — The system shall allow a student to create and edit a profile, including university, bio, availability, and experience level.

- FR-1.4 — The system shall allow a student to add, update, or remove skills from their profile, each with a self-rated proficiency level (Beginner, Intermediate, Advanced).
- FR-1.5 — The system shall allow a student to select one or more interest areas from a platform-managed catalog.
- FR-1.6 — The system shall require a complete profile (skills and interests recorded) before a student can create a team, send an invitation, or be added as a confirmed team member.

1.2.2 Team Formation (Leader-Driven, Invitation-Based)

- FR-2.1 — The system shall allow a student to create a new team; that student becomes the team's Team Leader.
- FR-2.2 — The system shall allow only the Team Leader to send invitations on behalf of their team.
- FR-2.3 — Each invitation shall specify a proposed role (e.g. Frontend, Backend, Database, UI/UX, QA) for the invited student.
- FR-2.4 — The system shall allow an invited student to accept or decline an invitation.
- FR-2.5 — Accepting an invitation shall create a confirmed team membership with the role proposed in that invitation. Declining shall close the invitation with no membership created.
- FR-2.6 — The system shall allow a Team Leader to withdraw a pending invitation before it has been responded to.
- FR-2.7 — The system shall allow a Team Leader to request AI-suggested candidate teammates, based on the matching between candidate skills and the team's stated requirements.
- FR-2.8 — Each AI-suggested candidate shall include a suggested role and a match score.
- FR-2.9 — AI Team Matching shall never create an invitation or a team membership automatically. A Team Leader must explicitly convert a suggestion into a real invitation, which still follows the normal accept/decline lifecycle (FR-2.4, FR-2.5).
- FR-2.10 — The system shall allow a Team Leader to remove a confirmed member from the team.
- FR-2.11 — The system shall compute and display a Team Readiness Score reflecting the skill coverage of confirmed team members against the team's stated requirements.

1.2.3 Project Planning

- FR-3.1 — The system shall allow a team to request AI-generated project recommendations once it has confirmed members.
- FR-3.2 — Each project recommendation shall include a title, description, and confidence score reflecting fit with the team's combined skills and interests.
- FR-3.3 — The system shall cache generated recommendations so that revisiting them does not require a repeated AI call.
- FR-3.4 — The system shall allow a team to accept one recommendation, which creates the team's active project.

- FR-3.5 — The system shall allow a team to generate an AI roadmap for its active project once a project has been accepted.
- FR-3.6 — A generated roadmap shall be broken into an ordered sequence of phases (e.g. Requirements Analysis, System Design, Development, Testing, Deployment).

1.2.4 Project Management

- FR-4.1 — The system shall allow a team member to create a task within a roadmap phase, with a title, description, and deadline.
- FR-4.2 — The system shall allow a task to be assigned to a specific confirmed team member.
- FR-4.3 — The system shall allow a task's status to be updated through the lifecycle: to do → in progress → done.
- FR-4.4 — Marking a task as done shall trigger a reputation update for the assigned member.
- FR-4.5 — The system shall allow a team member to opt in to a team via an “I Commit” action, and to later withdraw that commitment.
- FR-4.6 — The system shall maintain an immutable log of all reputation-affecting events per student (task completions, missed deadlines, commitments, withdrawals) and compute a displayed score from that log.

1.2.5 Administration

- FR-5.1 — The system shall allow an Admin to add, edit, or remove entries in the global skills catalog.
- FR-5.2 — The system shall allow an Admin to add, edit, or remove entries in the global interests catalog.
- FR-5.3 — The system shall allow an Admin to add, edit, or remove entries in the global roles catalog.

1.3 User Stories

User stories are written in the standard “As a …, I want …, so that …” format and grouped by the same functional areas as the requirements above. Each story maps to one or more functional requirements (FR) listed in Section 1.2.

1.3.1 Management

#	User Story	Maps to
US-1	As a new student, I want to register an account with my email and password, so that I can access the platform.	FR-1.1
US-2	As a returning student, I want to log in securely, so that I can resume working with my team.	FR-1.2
US-3	As a student, I want to build a profile with my skills and interests, so that the AI and other students can evaluate whether I'm a good fit for a team.	FR-1.3, FR-1.4, FR-1.5
US-4	As a student, I want to be prevented from creating or joining a team until my profile is complete, so that team matching and readiness scoring are always based on accurate data.	FR-1.6

1.3.2 Team Formation

#	User Story	Maps to
US-5	As a student, I want to create a team and automatically become its leader, so that I have control over who joins and in what role.	FR-2.1
US-6	As a Team Leader, I want to invite a specific student into a specific role, so that I can build the team I have in mind.	FR-2.2, FR-2.3
US-7	As an invited student, I want to see the role being offered before I decide, so that I can accept or decline with full information.	FR-2.3, FR-2.4
US-8	As an invited student, I want to accept or decline an invitation, so that I only join teams and roles I actually want.	FR-2.4, FR-2.5
US-9	As a Team Leader, I want to withdraw an invitation I no longer want to extend, so that I can correct a mistake before the candidate responds.	FR-2.6
US-10	As a Team Leader who doesn't have specific teammates in mind, I want the AI to suggest compatible candidates and a role for each, so that I don't have to manually search through every student's profile.	FR-2.7, FR-2.8
US-11	As a Team Leader, I want AI suggestions to never join my team automatically, so that I always retain the final decision over who is on my team.	FR-2.9
US-12	As a Team Leader, I want to remove a member who is no longer participating, so that I can keep the team and its readiness score accurate.	FR-2.10
US-13	As a Team Leader, I want to see a readiness score for my team, so that I know whether we have the skill coverage needed before starting a project.	FR-2.11

1.3.3 Project Planning

#	User Story	Maps to
US-14	As a Team Leader, I want the AI to suggest projects that fit my team's combined skills and interests, so that we don't waste time brainstorming from scratch.	FR-3.1, FR-3.2
US-15	As a Team Leader, I want to revisit previously suggested projects without waiting for the AI again, so that comparing options is fast.	FR-3.3
US-16	As a team, we want to accept one recommended project as our active project, so that the rest of the platform (tasks, roadmap) has something concrete to organize around.	FR-3.4
US-17	As a team, we want an AI-generated roadmap for our chosen project, so that we have a structured starting plan instead of an empty task board.	FR-3.5, FR-3.6

1.3.4 Project Management

#	User Story	Maps to
US-18	As a team member, I want to create a task under a roadmap phase, so that the work needed to complete that phase is tracked.	FR-4.1
US-19	As a team member, I want to assign a task to a specific teammate, so that responsibility for each task is clear.	FR-4.2
US-20	As a team member, I want to update a task's status as I work on it, so that the rest of the team can see real-time progress.	FR-4.3
US-21	As a team member, I want completing tasks on time to improve my reputation score, so that my reliability is reflected and visible to future teammates.	FR-4.4, FR-4.6
US-22	As a team member, I want to formally commit to a team, and to withdraw that commitment if my circumstances change, so that the team has an honest signal of who is actually participating.	FR-4.5

1.3.5 Administration

#	User Story	Maps to
US-23	As an Admin, I want to manage the global skills, interests, and roles catalogs, so that matching, recommendations, and invitations always draw from a clean, consistent set of values.	FR-5.1, FR-5.2, FR-5.3

1.4 Notes on Alignment with Later Chapters

- **Invitation as the only path to membership:** FR-2.4, FR-2.5, and FR-2.9 are implemented at the database level in Chapter 5 (TEAM_INVITATIONS as the sole gatekeeper of TEAM_MEMBERS) and at the API level in Chapter 4 (the Invitation API and the suggestion-only AI Team Matching API).
- **Team-first, then project:** FR-2.1 through FR-2.11 (team formation) are listed before FR-3.1 through FR-3.6 (project planning) deliberately — a team must exist and be staffed before the AI generates project recommendations for it, consistent with the use case diagram and user flow in Chapter 3.
- **AI as suggestion-only throughout:** FR-2.9 (team matching) and the recommendation-then-accept structure of FR-3.1/FR-3.4 (project planning) both follow the same principle — the AI System proposes, a human actor (the Team Leader, or the team collectively) disposes.

CHAPTER 2

Non-Functional Requirements, Project Scope & Technology Stack

Member 2 — Quality & Scope Lead

2.1 Non-Functional Requirements

2.1.1 Performance

- The system should load pages within 3 seconds under normal operating conditions.
- Team matching results should be generated within 5 seconds.
- API responses should be returned within 2 seconds for standard requests.
- The platform should support at least 500 concurrent users.

2.1.2 Security

- User authentication should be implemented using JWT.
- Passwords must be encrypted before storage.
- All communication should be secured using HTTPS.
- Access to sensitive resources should be restricted based on user roles and permissions.

2.1.3 Reliability

- The platform should maintain an uptime of at least 99%.
- User data should be stored persistently in PostgreSQL.
- System failures should not result in data loss.
- Database backups should be performed regularly.

2.1.4 Scalability

- The architecture should support future feature expansion.
- The system should accommodate increasing numbers of users and projects.
- Additional AI modules and integrations should be added without major redesign.

2.1.5 Usability

- Users should be able to create a profile within a few minutes.
- Navigation should be simple and consistent across the platform.
- The system should provide clear feedback for user actions.
- The interface should be responsive and user-friendly.

2.1.6 Maintainability

- The application should follow a modular architecture.
- Source code should be managed using GitHub.
- Features should be developed through separate branches.
- Documentation should be maintained for future developers.

2.2 Project Scope (MVP Features)

2.2.1 Included Features

User Management

- User Registration
- User Login
- Profile Creation
- Skills and Interests Management

Team Formation

- Team Creation (by a Team Leader)
- Invitation-Based Member Recruitment
- AI-Assisted Team Matching (suggestion-only)
- Role Assignment at Invitation

Project Planning

- AI Project Recommendations
- Project Readiness Score
- AI Roadmap Generation

Project Management

- Task Creation
- Task Assignment
- Task Status Tracking

Accountability Features

- Reputation System

2.2.2 Excluded Features

The following features are outside the scope of the MVP and may be implemented in future versions:

- Real-Time Chat System
- GitHub Integration
- Advanced Analytics Dashboard
- AI Risk Detection
- Mobile Application
- Video Conferencing
- Notification System
- Team Performance Analytics

2.3 Technology Stack Justification

2.3.1 Frontend

React

Purpose: User Interface Development

- Component-based architecture
- Reusable UI components
- Fast development process
- Strong community support

Tailwind CSS

Purpose: Styling and Responsive Design

- Rapid UI development
- Consistent design system
- Responsive layouts
- Easy maintenance

2.3.2 Backend

FastAPI

Purpose: API and Business Logic Development

- High performance
- Automatic API documentation
- Easy AI integration
- Modern Python framework

2.3.3 Database

PostgreSQL

Purpose: Data Storage

- Reliable relational database
- Strong support for complex relationships
- High performance and scalability
- ACID compliance for data integrity

2.3.4 Authentication

JWT (JSON Web Token)

Purpose: User Authentication and Authorization

- Secure authentication mechanism
- Stateless architecture
- Easy integration with frontend and backend systems

2.3.5 AI Components

LLM API Integration

Purpose: Project Recommendations, Team Matching Suggestions, and Roadmap Generation

- Generates intelligent project suggestions
- Suggests compatible teammates based on skills and project requirements
- Creates structured project roadmaps
- Enhances team decision-making processes

2.3.6 Deployment

Docker (Optional)

Purpose: Application Deployment and Environment Management

- Consistent development environments
- Simplified deployment process
- Improved scalability and portability

CHAPTER 3

Use Case Diagram & User Flow / Workflow

Member 3 — Use Case & Workflow Lead

This chapter defines the system actors and use cases for the CollabIQ AI platform, and documents the end-to-end user workflow across two stages: (1) onboarding and team formation, and (2) project planning and task execution. Team formation follows an invitation-based model: a Team Leader creates a team and invites specific members into proposed roles, optionally guided by AI-suggested candidates. The use case diagram identifies who interacts with the system and what they can do; the workflow diagrams trace the exact screen-by-screen path a student follows from first visiting the platform to having a fully formed team with an assigned roadmap and tracked tasks.

3.1 System Actors

Three actors interact with CollabIQ, matching the actors implied by the system architecture and API design (Chapter 4):

Actor	Type	Description
Student	Primary, human	The end user of the platform. Registers, builds a profile, creates or is invited into a team, and works through tasks and roadmap phases. A student who creates a team becomes its Team Leader.
AI System	Secondary, system	The backend AI service responsible for skill matching, project recommendations, and roadmap generation. AI Team Matching only produces suggestions — it never adds a member to a team directly. Never interacts with the student except through the Application Layer.
Admin	Secondary, human	Maintains the platform's reference data: the global skills catalog and the role catalog used by the matching and invitation process.

3.2 Use Case Diagram

The diagram below groups use cases under four functional areas — Management, Team Formation, Project Planning, and Project Management — inside the CollabIQ System boundary. Create Profile includes Manage Skills, since a profile is not considered complete until skills are recorded. Smart Team Matching includes Auto Role Assignment, reflecting that each AI-suggested candidate comes with a suggested role attached. The Student actor drives Registration, Login, team formation, and project management actions; the AI System participates in Auto Role Assignment, Smart Team Matching, and AI Roadmap Generation; the Admin actor is positioned to manage the platform's reference catalogs.

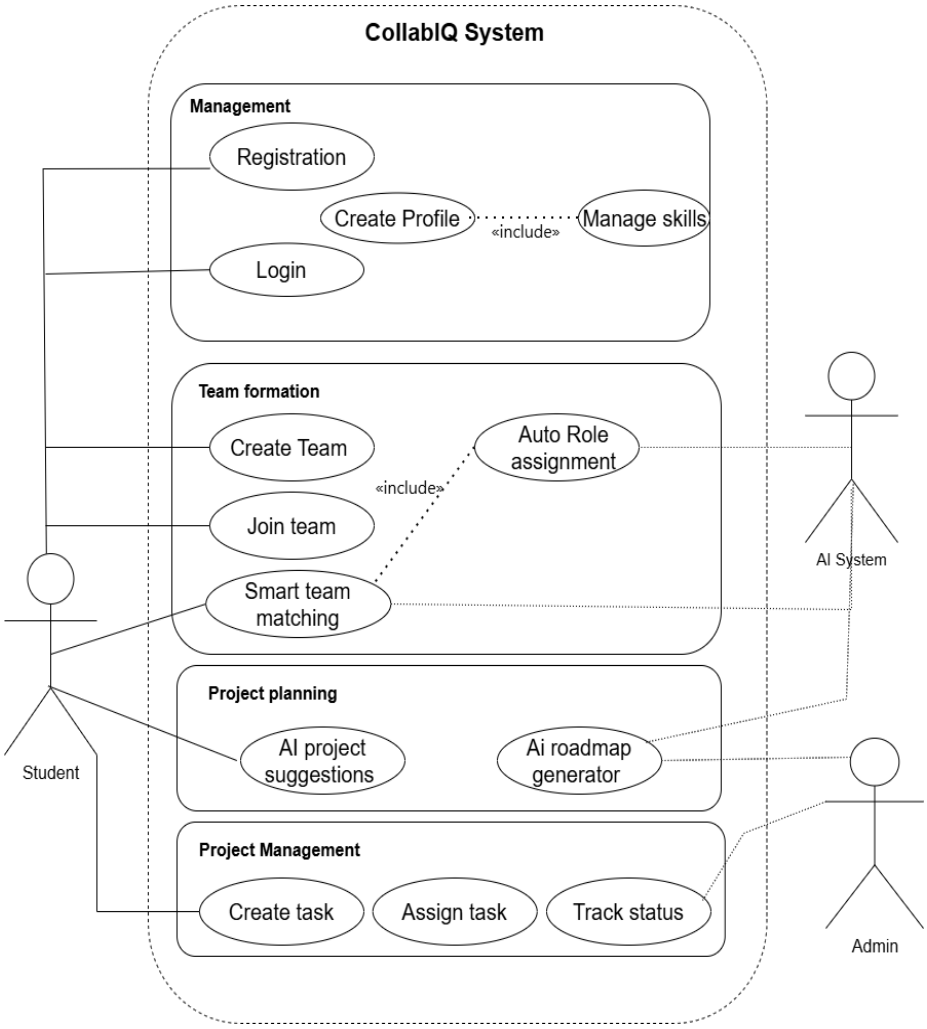


Figure 3.1: Use Case Diagram — CollabIQ System

3.3 Use Case Descriptions

Use case	Actor(s)	Description
Registration	Student	Create a new account with name, email, and password. Entry point for new users on the landing page.
Login	Student	Authenticate with email and password to receive a JWT access token. Entry point for returning users.
Create Profile	Student	Complete personal profile information required before creating or being invited into a team. Includes Manage Skills.
Manage skills	Student	Select skills (with proficiency level) and interest areas from the platform catalog.
Create Team	Student	Start a new team and become its Team Leader. The leader can then send invitations or request AI-suggested candidates.
Join team	Student	Accept or decline an invitation sent by a Team Leader for a proposed role.
Smart team matching	Student, AI System	The Team Leader requests AI-suggested candidates based on skills and project requirements. Includes Auto Role Assignment — each suggestion carries a proposed role. The AI only suggests; it does not add members automatically.
Auto Role assignment	AI System	Attaches a suggested role to each AI-recommended candidate, based on the gap between the team's current skills and the project's requirements.
AI project suggestions	Student, AI System	Produces a ranked list of candidate projects for the team based on its combined skills and interests.
AI roadmap generator	AI System	Once the team selects a recommended project, generates an ordered set of roadmap phases (e.g. Requirements, Design, Development, Testing, Deployment).
Create task	Student	Add a task to a roadmap phase with a title, description, and deadline.
Assign task	Student	Assign a created task to a specific team member.
Track status	Student	Move a task through its lifecycle: to do → in progress → done.

3.4 User Flow / Workflow

The user flow is documented in two stages, matching the two workflow diagrams produced during design: (1) onboarding and team formation, and (2) project planning and task execution.

3.4.1 Stage 1 — Onboarding and Team Formation

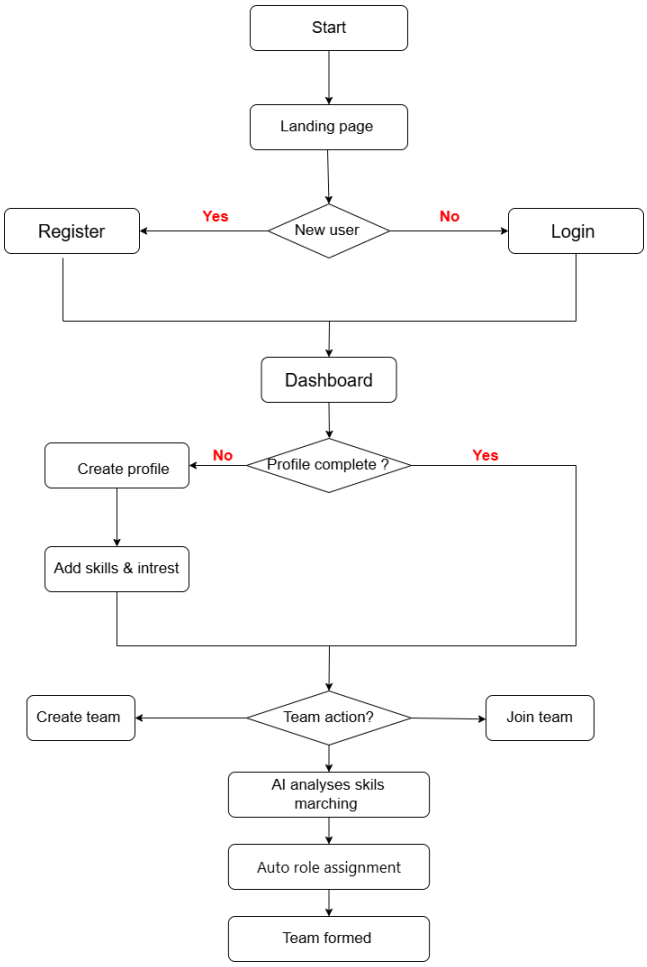


Figure 3.2: Stage 1 Workflow — Onboarding and Team Formation

This stage covers everything from a visitor's first interaction with the platform to having a fully formed, role-assigned team.

1. Start — the student opens the platform and lands on the landing page.
2. New user? — the system checks whether this is a first-time visitor.
    - Yes → Register: the student creates a new account.
    - No → Login: the student signs in with existing credentials.
3. Both paths converge on the Dashboard, the student's central hub.
4. Profile complete? — the system checks whether the student has a complete profile.
    - No → Create profile, then Add skills & interests, before continuing.
    - Yes → skip directly to team formation.
5. Team action? — the student chooses to either:
    - Create team — become the Team Leader of a new team, or
    - Join team — respond to an invitation received from a Team Leader.
6. AI analyses skill matching — if the Team Leader does not already have teammates in mind, they can request AI-suggested candidates based on skills and project requirements.
7. Auto role assignment — each AI-suggested candidate is shown with a proposed role based on the team's skill gaps.
8. Team formed — once enough invitations are accepted, the team is finalized and ready to move into project planning (Stage 2).

3.4.2 Stage 2 — Project Planning and Task Execution

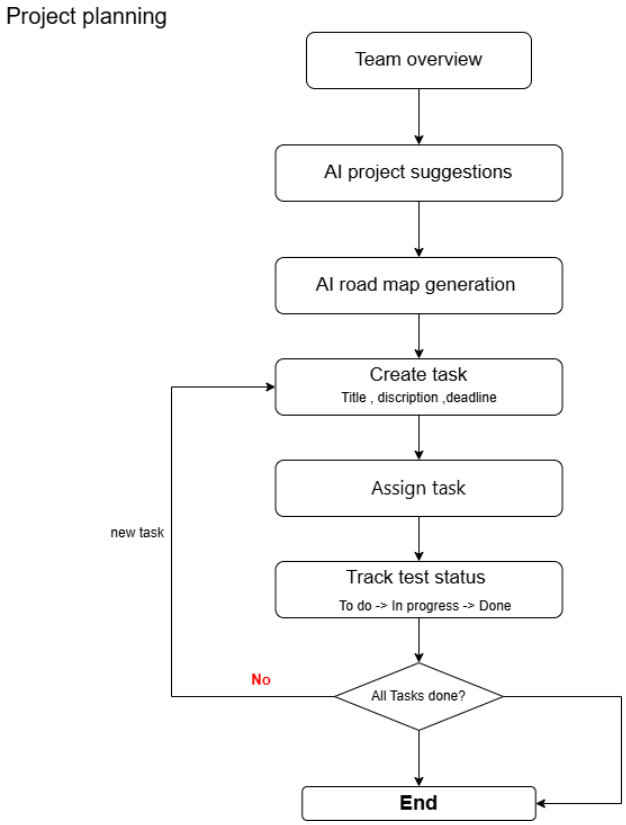


Figure 3.3: Stage 2 Workflow — Project Planning and Task Execution

This stage begins immediately after a team is formed and covers selecting a project, generating a roadmap, and managing tasks through to completion.

9. Team overview — the team reviews its members, roles, and readiness score before proceeding.
10. AI project suggestions — the AI engine generates a ranked list of candidate projects based on the team's combined skills and interests.
11. AI roadmap generation — once the team selects a project, the AI generates a structured roadmap broken into ordered phases.
12. Create task — a team member creates a task within a roadmap phase, specifying a title, description, and deadline.
13. Assign task — the task is assigned to a specific team member.
14. Track task status — the task moves through to do → in progress → done.
15. All tasks done? — the system checks whether every task in the current phase (and roadmap) is complete.
    - No → loop back to Create task for the next task.
    - Yes → proceed to End.
16. End — the project planning cycle for the current roadmap is complete.

3.5 Team Formation Process — Detailed Rules

Team formation in CollabIQ is leader-driven and invitation-based, not automatic. The exact rules are:

17. A Team Leader creates a team and sends invitations to the members they want to work with.
18. Each invitation includes the proposed role / position (Frontend, Backend, Database, UI/UX, QA, etc.) for the invited member.
19. Invited members can accept or decline the invitation.
20. If a user doesn't already have teammates in mind, they can use the AI Team Matching feature, which generates a list of compatible members based on their skills and the project's requirements.
21. The AI only suggests members — it does not automatically add them to the team. The Team Leader must still send a formal invitation to any AI-suggested candidate.

3.6 Workflow Notes

- **Two decision gates control entry into team formation:** “New user?” and “Profile complete?” both must resolve before a student can create a team or be invited into one, ensuring the AI matching engine always has complete skill and interest data to work with.
- **Team formation precedes project selection:** consistent with the database design in Chapter 5, the workflow confirms that a team is created and staffed first, and only afterward does the AI generate project recommendations for that team — not the reverse.
- **AI suggestions never bypass the invitation step:** whether a member joins through a direct invitation or an AI-suggested match, the same invitation → accept/decline lifecycle applies. The AI engine has no permission to add a member to a team on its own.
- **The task loop is the only cycle in the workflow:** “All tasks done?” routes back to “Create task” until every task in the roadmap is marked done, after which the flow reaches End. Every other decision point in the workflow is a one-time fork, not a loop.
- **Marking a task done has a side effect:** per the API design in Chapter 4, completing a task triggers a reputation update, linking the task-tracking workflow directly to the reputation and commitment system.

CHAPTER 4

System Architecture & API Planning

Member 4 — Architecture & Integration Lead

This chapter defines the technical architecture of the CollabIQ AI platform and the full REST API surface that connects the frontend, backend, database, and AI service. The architecture and endpoints below are aligned with the team-first, invitation-based workflow agreed by the group: a Team Leader creates a team and invites members into proposed roles — optionally guided by AI-suggested candidates — before any project exists. Only after the team is staffed does the AI engine generate project recommendations for that team, and a roadmap is generated only once the team accepts one of those recommendations. This is consistent with the use case diagram (Chapter 3) and the database design (Chapter 5).

4.1 System Architecture Overview

CollabIQ follows a layered client-server architecture that separates presentation, business logic, data persistence, and AI services into four distinct layers. The frontend communicates with the backend exclusively over HTTPS using JSON payloads secured by JWT bearer tokens. Only the backend communicates with the external AI service, using a server-side API key that is never exposed to the client.

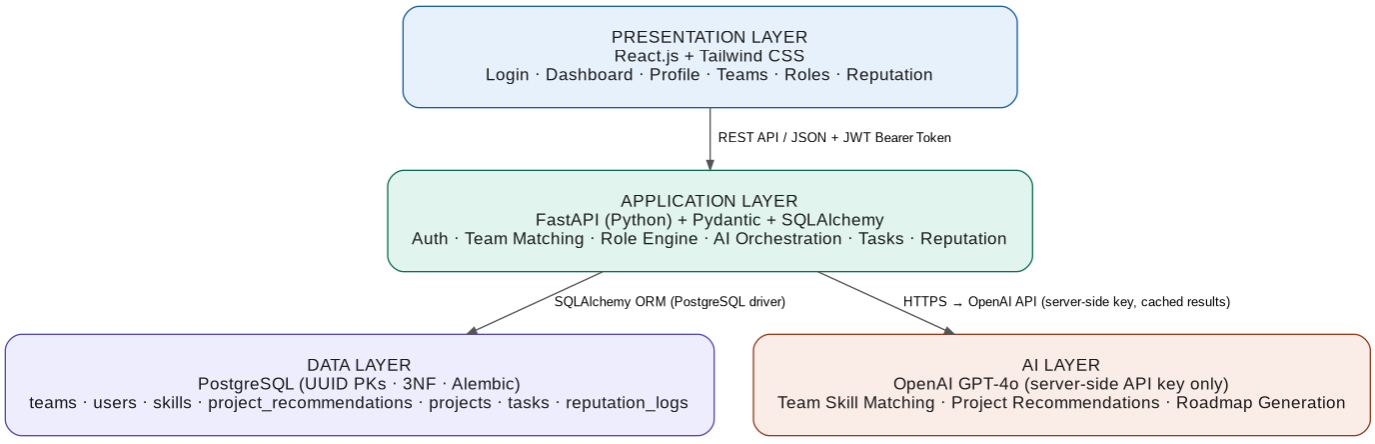


Figure 4.1: CollabIQ Four-Layer System Architecture

Layer	Technology	Responsibility
Presentation	React.js + Tailwind CSS	Renders all UI screens. Communicates with the backend via REST only. Stores the JWT in memory and attaches it to every protected request.
Application	FastAPI (Python) + Pydantic + SQLAlchemy	Holds all business logic in independent service modules. Pydantic validates every request body. Handles auth, team formation, invitations, AI-assisted matching, role assignment, AI orchestration, tasks, and reputation.
Data	PostgreSQL + SQLAlchemy + Alembic	Persistent relational storage, queried exclusively through the ORM (no raw SQL). Alembic manages schema versions and migrations.
AI	OpenAI GPT-4o (external)	Called only from the backend using a server-side API key loaded from environment variables. Results are cached in the database and never called directly from the frontend.

4.2 DFD Level 0 — Context Diagram

The context diagram treats CollabIQ as a single process. The Student is the only external entity; all inputs originate from the student and all outputs return to the student. This establishes the system boundary before the process is decomposed further in the Level 1 diagram.

Figure 4.2: DFD Level 0 — Context Diagram

4.3 DFD Level 1 — System Processes

The Level 1 diagram decomposes the system into six core processes and five data stores, and follows the team-first, invitation-based sequence: a student authenticates, a Team Leader creates a team and sends invitations (optionally informed by AI-suggested candidates), invited members accept or decline, and only once the team is staffed does the AI engine generate project recommendations for that team. Once the team accepts a recommendation, a roadmap is generated and the task / reputation loop begins.

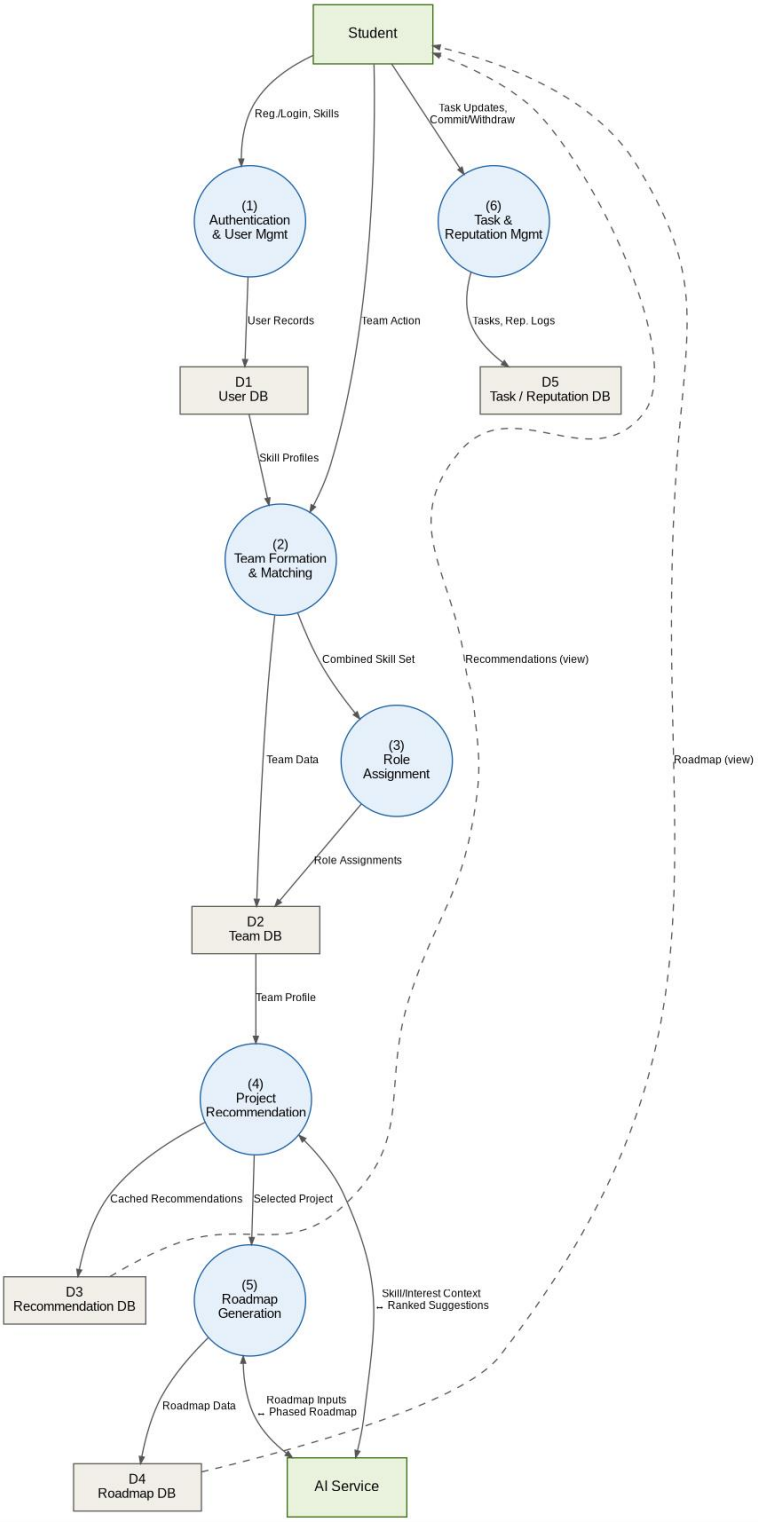


Figure 4.3: DFD Level 1 — System Processes

Process	Source	Inputs	Outputs / Data stores
(1) Authentication & User Mgmt	Student	Registration / login, skills, interests	User records → D1 User DB
(2) Team Formation & Invitations	Student / Internal	Team creation, invitations sent, accept/decline responses, optional AI match request	Team data, invitations → D2 Team DB; accepted members → Process 3
(3) Role Confirmation	Internal	Proposed role from the accepted invitation	Confirmed role assignments → D2 Team DB
(4) Project Recommendation	AI Service	Team profile from D2	Cached recommendations → D3 Recommendation DB → Student; selected project → Process 5
(5) Roadmap Generation	AI Service	Selected project from Process 4	Roadmap → D4 Roadmap DB → Student (view)
(6) Task & Reputation Mgmt	Student	Task updates, commit / withdraw actions	Tasks and reputation logs → D5 Task / Reputation DB

4.4 Integration Strategy

Integration point	Protocol	Details
Frontend ↔ Backend	HTTPS REST API + JWT	JSON payloads. JWT bearer token attached to every protected request. CORS restricted to the frontend origin. Token refresh at expiry via POST /auth/refresh. Logout adds the refresh token to a denylist.
Backend ↔ Database	SQLAlchemy ORM + PostgreSQL	All database operations go through ORM model classes; no raw SQL. Connection pooling supports concurrent requests. Alembic manages schema migrations with version-controlled scripts.
Backend ↔ AI Service	HTTPS → OpenAI REST API	A team's combined skill profile, interests, and (once selected) project context are sent as structured prompts. GPT-4o returns JSON candidate suggestions, project recommendations, or a phased roadmap. Results are cached in the database. Retries and fallback handle rate-limit errors.
API key security	.env + python-dotenv	OPENAI_API_KEY is loaded at server startup only. Never included in API responses, logs, or source code. .env is listed in .gitignore. Rotation recommended every 90 days.

4.5 API Planning — Endpoint Reference

Base URL: https://api.collabiq.app/v1 · Auth header: Authorization: Bearer <access_token> · Format: JSON request / response.

Success envelope: { status: "success", data: { … } } · Error envelope: { status: "error", message: "…", code: "…" }

4.5.1 Authentication API

Endpoint	Description
`POST /auth/register`	Register a new student: full_name, email, password, university, department
`POST /auth/login`	Log in → returns access_token (30 min), refresh_token (7 days), token_type, expires_in
`POST /auth/refresh`	Exchange a refresh token for a new access token
`POST /auth/logout`	Invalidate the refresh token (adds it to the denylist)
`GET /auth/me`	Return the currently authenticated student's profile — protected

4.5.2 User & Profile API

Endpoint	Description
`GET /users/{user_id}`	Get a student profile by ID
`PUT /users/{user_id}`	Update name, university, bio, availability, experience_level
`GET /users/{user_id}/skills`	List all skills for a student with proficiency levels
`POST /users/{user_id}/skills`	Add a skill: skill_id, skill_level (1=Beginner, 2=Intermediate, 3=Advanced)
`DELETE /users/{user_id}/skills/{id}`	Remove a skill from the profile
`GET /users/{user_id}/interests`	List all interest areas selected by the student
`GET /users/{user_id}/teams`	All teams a student belongs to or leads
`GET /users/{user_id}/invitations`	All pending invitations sent to this student
`GET /users/{user_id}/reputation`	Reputation score and full change history
`GET /users/{user_id}/tasks`	All tasks assigned to the student

4.5.3 Skills, Interests & Roles Catalog API

Endpoint	Description
`GET /skills`	Full global skill catalog
`POST /skills`	Add a skill to the catalog — admin only
`GET /interests`	Full global interest-area catalog
`POST /interests`	Add an interest area to the catalog — admin only
`GET /roles`	All available role definitions (Frontend, Backend, Database, UI/UX, QA, etc.)
`GET /roles/{role_id}`	Role name and full responsibilities list

4.5.4 Team Formation API

Readiness label: Excellent (≥90) · Good (75–89) · Fair (60–74) · Needs Work (<60)

Endpoint	Description
POST /teams	Create a new team. The creating student becomes its Team Leader: team_name, description
GET /teams/{id}	Team details, leader, member list, readiness score
GET /teams/{id}/members	All confirmed team members with their roles
DELETE /teams/{id}/members/{user_id}	Remove a confirmed member from the team — leader only
GET /teams/{id}/readiness	Readiness breakdown: skill_coverage, availability, role_coverage, label

4.5.5 Invitation API

This is the only path by which a student joins a team. AI-suggested candidates (Section 4.5.6) still require a formal invitation through these endpoints — a suggestion is never auto-converted into membership.

Endpoint	Description
POST /teams/{id}/invitations	Team Leader sends an invitation: invited_user_id, proposed_role_id
GET /teams/{id}/invitations	List all invitations sent by this team and their status — leader only
GET /invitations/{invitation_id}	Get a single invitation's details, including the proposed role
POST /invitations/{invitation_id}/accept	Invited student accepts — creates the confirmed TEAM_MEMBERS record with the proposed role
POST /invitations/{invitation_id}/decline	Invited student declines — invitation closed, no membership created
DELETE /invitations/{invitation_id}	Team Leader withdraws a pending invitation

4.5.6 AI Team Matching API

Suggestion-only: this API never creates a membership or an invitation by itself. The Team Leader must review the results and call the Invitation API (4.5.5) to act on any suggestion.

Endpoint	Description
POST /teams/{id}/match/generate	Team Leader requests AI-suggested candidates: required_skills[], count — calls the AI service server-side
GET /teams/{id}/match	Retrieve cached match suggestions for the team, each with suggested_role_id and match_score
POST /teams/{id}/match/{suggestion_id}/invite	Convenience endpoint: converts one AI suggestion directly into a real invitation (still requires accept/decline)

4.5.7 AI Project Recommendation API

Response includes: title, description, confidence_score (0.00–1.00). Results are cached in the project_recommendations table, keyed by team_id.

Endpoint	Description
`POST /teams/{id}/recommendations/generate`	Generate AI project ideas for this team — calls GPT-4o server-side using the team's combined skills and interests
`GET /teams/{id}/recommendations`	Retrieve cached recommendations for the team (avoids repeat LLM calls)
`POST /teams/{id}/recommendations/{recommendation_id}/accept`	Team accepts a recommendation — creates the PROJECTS record linked to this team and recommendation
`DELETE /recommendations/{recommendation_id}`	Dismiss / remove a recommendation

4.5.8 Projects API

Endpoint	Description
`GET /projects/{id}`	Project details, including the team and recommendation it originated from
`PUT /projects/{id}`	Update title, description, or status
`DELETE /projects/{id}`	Delete a project — team leader only

4.5.9 Roadmap API

Default phases: Requirements Analysis → System Design → Development → Testing → Deployment

Endpoint	Description
`POST /roadmaps/generate`	Generate an AI roadmap for a project once it has been accepted: project_id — structured GPT-4o prompt
`GET /roadmaps/{project_id}`	Full roadmap with phases in order_no sequence
`PUT /roadmaps/{roadmap_id}/phases/{phase_id}`	Update phase status: todo → in_progress → done

4.5.10 Task Management API

Endpoint	Description

`POST /tasks`	Create a task: phase_id, title, description, assigned_to, due_date, status
`GET /tasks`	List tasks — filter by phase_id, project_id, or assigned_to
`GET /tasks/{id}`	Get a single task by ID
`PUT /tasks/{id}`	Update title, description, assignee, due date, or status
`DELETE /tasks/{id}`	Delete a task
`PATCH /tasks/{id}/status`	Quick status update: { status: "done" } — auto-triggers a reputation update

4.5.11 Reputation & Commitment API

Endpoint	Description
`GET /reputation/{user_id}`	Full score (0–100) and breakdown: commitment, task_completion, team_collaboration, punctuality
`GET /reputation/{user_id}/logs`	Full audit log of all reputation changes with timestamps
`POST /teams/{id}/commit`	Student commits to the team via “I Commit” — awards positive reputation points
`POST /teams/{id}/withdraw`	Student withdraws commitment — deducts reputation points

4.5.12 HTTP Status Code Reference

Code	Status	When it occurs
200	OK	Successful GET, PUT, PATCH
201	Created	Successful POST — resource created, ID returned in data
204	No Content	Successful DELETE — no body returned
400	Bad Request	Malformed JSON or invalid field value
401	Unauthorized	Missing, expired, or invalid JWT token
403	Forbidden	Valid token but insufficient permissions (e.g. non-leader trying to send an invitation)
404	Not Found	Requested resource does not exist
409	Conflict	Duplicate resource, or invitation already responded to
422	Unprocessable Entity	Pydantic validation failure — field-level error array returned
429	Too Many Requests	Rate limit exceeded — 100 req/min on auth and AI endpoints
500	Internal Server Error	Unhandled exception
503	Service Unavailable	OpenAI provider timeout or PostgreSQL database unreachable

4.6 Security Considerations

Security area	Implementation

Authentication	JWT HS256 tokens — 30-minute access token, 7-day refresh token. Refresh tokens are stored in a denylist on logout.
Password security	bcrypt hashing, cost factor 12. Plaintext passwords are never stored, logged, or transmitted.
Input validation	Pydantic schemas enforce type, length, regex, and format on every request body before processing.
Authorization	Role-Based Access Control (RBAC) — a role claim is embedded in the JWT payload. Team-leader-only actions (sending invitations, removing members) are additionally checked against the team's leader_id.
CORS policy	Whitelist of the frontend origin only; all other origins are rejected at the framework level.
AI API key	Stored in the server .env file only. Never in the database, API responses, or logs. .env is listed in .gitignore.
SQL injection	SQLAlchemy ORM is used exclusively across all repository-layer code. No raw SQL strings exist.
Rate limiting	100 req/min on authentication endpoints, 20 req/min on AI endpoints.

4.7 Frontend Screen ↔ API Integration Map

This table shows which endpoints each frontend screen calls, so frontend and backend work can proceed in parallel.

Frontend screen	Primary APIs called
Login / Register	POST /auth/login, POST /auth/register
Dashboard	GET /auth/me, GET /users/{id}/teams, GET /users/{id}/invitations, GET /tasks, GET /reputation/{id}
Profile	GET /users/{id}, PUT /users/{id}, GET/POST/DELETE /users/{id}/skills, GET /users/{id}/interests
Build New Team	POST /teams, GET /teams/{id}, GET /teams/{id}/readiness
AI Team Matching	POST /teams/{id}/match/generate, GET /teams/{id}/match, POST /teams/{id}/match/{id}/invite
Invitations / Team Management	POST /teams/{id}/invitations, GET /teams/{id}/invitations, POST /invitations/{id}/accept, POST /invitations/{id}/decline, GET /teams/{id}/members
Project Recommendations	POST /teams/{id}/recommendations/generate, GET /teams/{id}/recommendations, POST /teams/{id}/recommendations/{id}/accept
Roadmap Viewer	POST /roadmaps/generate, GET /roadmaps/{project_id}, PUT /roadmaps/{roadmap_id}/phases/{phase_id}
Task Board	POST /tasks, GET /tasks, PUT /tasks/{id}, PATCH /tasks/{id}/status, DELETE /tasks/{id}
Reputation & Commitment	GET /reputation/{id}, GET /reputation/{id}/logs, POST /teams/{id}/commit, POST /teams/{id}/withdraw

4.8 Scalability Roadmap

- Docker containerization — each service (API, database, Redis, worker) runs in an isolated container; docker-compose for local development, Kubernetes-ready for production.
- Horizontal backend scaling — stateless FastAPI servers behind a load balancer; multiple replicas can run concurrently since all state lives in PostgreSQL and Redis.
- Redis caching — reduces repeat OpenAI calls and latency; also stores the session token denylist for fast lookup.
- Background task queue — AI-heavy operations (match generation, recommendation generation, roadmap creation) offloaded to async workers so API responses are not blocked.
- Planned: WebSocket real-time updates for live invitation responses, task board changes, and notifications.
- Planned: GitHub integration to link roadmap milestones to repositories and auto-update task status from commits.

4.9 Notes on Alignment with Team Decisions

- **Invitation-based team formation:** membership is only ever created through POST /invitations/{id}/accept. There is no endpoint that adds a user to a team directly, including AI-driven ones — even the convenience endpoint in 4.5.6 still produces an invitation that requires acceptance.
- **AI Team Matching is read-only with respect to team state:** POST /teams/{id}/match/generate and GET /teams/{id}/match never write to TEAM_MEMBERS; they only populate the MATCH_SUGGESTIONS cache described in Chapter 5.
- **Consistency with the database design:** this matches the schema in Chapter 5, where TEAMS has a leader_id, TEAM_INVITATIONS carries the proposed role and status, and TEAM_MEMBERS is only ever created from an accepted invitation.
- **Consistency with the use case diagram:** the «include» relationship from Smart Team Matching to Auto Role Assignment is implemented by every match suggestion carrying a suggested_role_id, which becomes the proposed_role_id of an invitation if the leader acts on it.

CHAPTER 5

Database Design (ERD)

Member 5 — Data Lead

This chapter presents the database design for the CollabIQ AI platform. The design translates the functional requirements (Chapter 1), the non-functional requirements and MVP scope (Chapter 2), and the use case diagram and user flow (Chapter 3) into a relational schema implemented in PostgreSQL. The schema implements an invitation-based, leader-driven team formation model: a Team Leader creates a team and sends invitations carrying a proposed role to specific members; invited members accept or decline; and AI Team Matching only produces cached suggestions that the leader may turn into real invitations — it never writes a membership record directly.

5.1 Design Approach

The schema follows standard relational database principles and is normalized to Third Normal Form (3NF) to eliminate redundancy and maintain data integrity. Each entity represents a distinct concept in the system, and many-to-many relationships are resolved through dedicated junction tables. All primary keys use UUIDs rather than auto-incrementing integers, which supports distributed ID generation on the FastAPI backend and avoids exposing sequential, guessable identifiers through the API. Team membership is deliberately modeled as the result of an approved workflow (an accepted invitation) rather than a simple join table, since the business rule — “no one joins a team without the leader's approval” — must be enforceable at the data level, not just in application code.

5.2 Entity Relationship Diagram

The diagram below shows all fifteen entities in the CollabIQ database along with their primary keys (PK), foreign keys (FK), and the cardinality of each relationship.

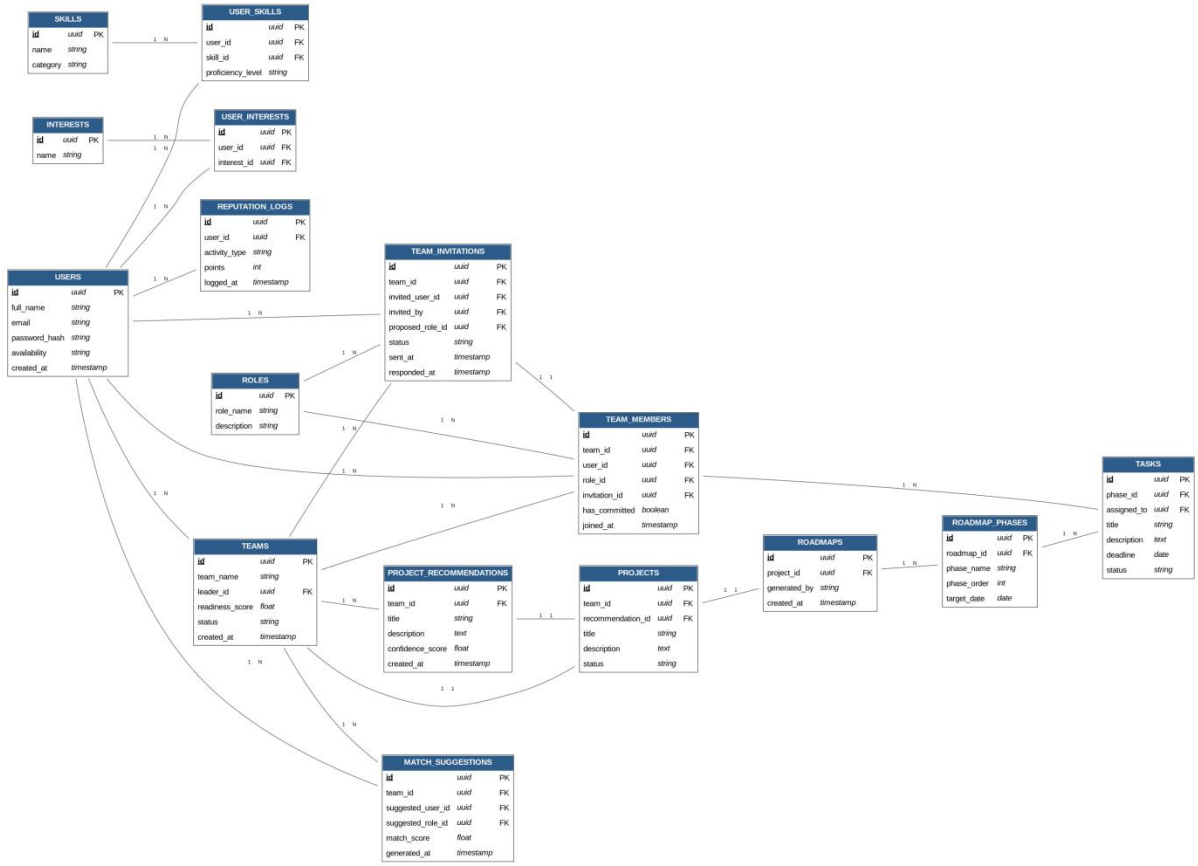


Figure 5.1: Entity Relationship Diagram (ERD) — CollabIQ AI Database

5.3 Entity Descriptions

The following subsections describe each table in the schema, its purpose within the platform, its fields, and its relationships to other tables.

5.3.1 USERS

Stores core account and profile information for every student on the platform. This is the central entity referenced by nearly every other table, either directly or through a junction table.

Field	Type	Key	Description
id	*uuid*	PK	Unique identifier for the user
full_name	*string*		Student's full name
email	*string*		Unique login credential, used for authentication
password_hash	*string*		Encrypted password (never stored in plain text)
availability	*string*		Self-reported weekly availability (e.g. part-time, full-time)
created_at	*timestamp*		Account creation date, used for reputation/activity tracking

5.3.2 SKILLS

A reference catalog of skills that students can select from when building their profile (e.g. backend development, UI/UX design, database management). Kept as a separate table rather than free text so the AI matching engine can compare skill sets reliably.

Field	Type	Key	Description
id	*uuid*	PK	Unique identifier for the skill
name	*string*		Name of the skill (e.g. “React”, “FastAPI”)
category	*string*		Grouping used for role suggestions (frontend, backend, database, UI/UX, QA)

5.3.3 USER_SKILLS

Junction table resolving the many-to-many relationship between users and skills. Each row represents one skill claimed by one user, along with a self-rated proficiency level.

Field	Type	Key	Description
id	*uuid*	PK	Unique identifier for the record
user_id	*uuid*	FK	References USERS.id
skill_id	*uuid*	FK	References SKILLS.id
proficiency_level	*string*		Beginner, intermediate, or advanced

5.3.4 INTERESTS

A reference catalog of project interest areas (e.g. fintech, healthcare, gaming), used by the AI recommendation engine to suggest relevant project ideas to a formed team.

Field	Type	Key	Description
id	*uuid*	PK	Unique identifier for the interest area
name	*string*		Name of the interest area

5.3.5 USER_INTERESTS

Junction table resolving the many-to-many relationship between users and interest areas.

Field	Type	Key	Description
id	*uuid*	PK	Unique identifier for the record
user_id	*uuid*	FK	References USERS.id
interest_id	*uuid*	FK	References INTERESTS.id

5.3.6 ROLES

A reference catalog of project roles that can be proposed in an invitation or suggested by AI matching (backend, frontend, database, UI/UX, QA, team lead).

Field	Type	Key	Description
id	*uuid*	PK	Unique identifier for the role
role_name	*string*		Name of the role (e.g. “Backend Developer”)
description	*string*		Short description of role responsibilities

5.3.7 TEAMS

Represents a project team. A team is always created by exactly one student, who becomes its leader and is the only one permitted to send invitations on the team's behalf. Stores the computed Project Readiness Score described in the MVP feature set.

Field	Type	Key	Description
id	*uuid*	PK	Unique identifier for the team
team_name	*string*		Display name of the team
leader_id	*uuid*	FK	References USERS.id — the student who created the team and leads it
readiness_score	*float*		Percentage score reflecting skill coverage across confirmed members
status	*string*		Forming, active, or completed
created_at	*timestamp*		Date the team was created

5.3.8 TEAM_INVITATIONS

Records every invitation sent by a Team Leader, whether sent directly or as the result of acting on an AI match suggestion. This table is the single gatekeeper of team membership: a row in TEAM_MEMBERS can only be created once the corresponding invitation here has been accepted.

Field	Type	Key	Description
id	*uuid*	PK	Unique identifier for the invitation
team_id	*uuid*	FK	References TEAMS.id
invited_user_id	*uuid*	FK	References USERS.id — the student being invited
invited_by	*uuid*	FK	References USERS.id — the team leader who sent the invitation
proposed_role_id	*uuid*	FK	References ROLES.id — the role offered in this invitation
status	*string*		Pending, accepted, or declined
sent_at	*timestamp*		Date the invitation was sent
responded_at	*timestamp*		Date the invited student accepted or declined (null while pending)

5.3.9 TEAM_MEMBERS

Represents confirmed team membership. Every row is created from — and links back to — exactly one accepted TEAM_INVITATIONS record, so the role a member holds is always traceable to the invitation that proposed it.

Field	Type	Key	Description
id	*uuid*	PK	Unique identifier for the membership record
team_id	*uuid*	FK	References TEAMS.id
user_id	*uuid*	FK	References USERS.id
role_id	*uuid*	FK	References ROLES.id — copied from the accepted invitation's proposed_role_id
invitation_id	*uuid*	FK	References TEAM_INVITATIONS.id — the accepted invitation that created this membership
has_committed	*boolean*		Whether the member opted in with “I Commit”
joined_at	*timestamp*		Date the invitation was accepted and membership created

5.3.10 MATCH_SUGGESTIONS

Caches AI Team Matching results for a team: candidate students who are not yet members, each paired with a suggested role and a match score. This table is read-only with respect to team membership — nothing here ever becomes a TEAM_MEMBERS row without first passing through a TEAM_INVITATIONS record that the candidate accepts.

Field	Type	Key	Description
id	*uuid*	PK	Unique identifier for the suggestion
team_id	*uuid*	FK	References TEAMS.id — the team this candidate was suggested for
suggested_user_id	*uuid*	FK	References USERS.id — the candidate student
suggested_role_id	*uuid*	FK	References ROLES.id — the role the AI believes fits this candidate
match_score	*float*		Model's confidence that this candidate fits the team's skill gap (0.0–1.0)
generated_at	*timestamp*		Date the suggestion was generated

5.3.11 PROJECT_RECOMMENDATIONS

Caches AI-generated project suggestions for a team, generated after the team has been staffed and its combined skill profile is known. Each recommendation includes a confidence score.

Field	Type	Key	Description
id	*uuid*	PK	Unique identifier for the recommendation
team_id	*uuid*	FK	References TEAMS.id
title	*string*		Suggested project title
description	*text*		AI-generated project description
confidence_score	*float*		Model's confidence that this project fits the team's skills (0.0–1.0)
created_at	*timestamp*		Date the recommendation was generated

5.3.12 PROJECTS

Stores the project a team is actively working on. A project only exists once a team has selected one of its AI-generated recommendations.

Field	Type	Key	Description
id	*uuid*	PK	Unique identifier for the project
team_id	*uuid*	FK	References TEAMS.id
recommendation_id	*uuid*	FK	References PROJECT_RECOMMENDATIONS.id — the suggestion the team accepted
title	*string*		Project title
description	*text*		Project description / summary
status	*string*		Accepted, in progress, or completed

5.3.13 ROADMAPS

Represents the AI-generated roadmap for a project. Kept as a separate entity from PROJECTS so it can be broken down into structured, trackable phases.

Field	Type	Key	Description
id	*uuid*	PK	Unique identifier for the roadmap
project_id	*uuid*	FK	References PROJECTS.id
generated_by	*string*		Source of the roadmap (AI-generated or manually created)
created_at	*timestamp*		Date the roadmap was generated

5.3.14 ROADMAP_PHASES

Breaks a roadmap into ordered phases or milestones (e.g. Requirements Analysis, System Design, Development, Testing, Deployment).

Field	Type	Key	Description
id	*uuid*	PK	Unique identifier for the phase
roadmap_id	*uuid*	FK	References ROADMAPS.id
phase_name	*string*		Name of the phase or milestone
phase_order	*int*		Sequence position of the phase within the roadmap
target_date	*date*		Planned completion date for the phase

5.3.15 TASKS

Individual, assignable units of work within a roadmap phase, matching the “Create task → Assign task → Track task status” loop shown in the project planning user flow.

Field	Type	Key	Description
id	*uuid*	PK	Unique identifier for the task
phase_id	*uuid*	FK	References ROADMAP_PHASES.id
assigned_to	*uuid*	FK	References TEAM_MEMBERS.id
title	*string*		Task title
description	*text*		Task description
deadline	*date*		Task deadline
status	*string*		To do, in progress, or done

5.3.16 REPUTATION_LOGS

Records individual reputation-affecting events for a user. The user's overall reputation score is calculated as an aggregate of this table rather than stored as a single mutable field, preserving a full audit trail.

Field	Type	Key	Description
id	*uuid*	PK	Unique identifier for the log entry
user_id	*uuid*	FK	References USERS.id
activity_type	*string*		Type of activity (task completed, deadline missed, commitment honored, etc.)
points	*int*		Reputation points awarded or deducted for this activity
logged_at	*timestamp*		Timestamp of the activity

5.4 Relationship Summary

- USERS (1) — (N) USER_SKILLS, USER_INTERESTS, REPUTATION_LOGS
- SKILLS (1) — (N) USER_SKILLS
- INTERESTS (1) — (N) USER_INTERESTS
- USERS (1) — (N) TEAMS (as leader_id — a student may lead several teams over time)
- TEAMS (1) — (N) TEAM_INVITATIONS
- USERS (1) — (N) TEAM_INVITATIONS (as invited_user_id and as invited_by)
- ROLES (1) — (N) TEAM_INVITATIONS (as proposed_role_id)
- TEAM_INVITATIONS (1) — (1) TEAM_MEMBERS (an accepted invitation produces exactly one membership)
- TEAMS (1) — (N) TEAM_MEMBERS
- ROLES (1) — (N) TEAM_MEMBERS
- TEAMS (1) — (N) MATCH_SUGGESTIONS
- USERS (1) — (N) MATCH_SUGGESTIONS (as suggested_user_id)
- TEAMS (1) — (N) PROJECT_RECOMMENDATIONS
- TEAMS (1) — (1) PROJECTS
- PROJECT_RECOMMENDATIONS (1) — (1) PROJECTS (the specific suggestion the team accepted)
- PROJECTS (1) — (1) ROADMAPS
- ROADMAPS (1) — (N) ROADMAP_PHASES
- ROADMAP_PHASES (1) — (N) TASKS
- TEAM_MEMBERS (1) — (N) TASKS (a task is assigned to one team membership)

5.5 Normalization Notes

- **1NF:** All attributes hold atomic values; multi-valued attributes such as a user's skills or interests are moved into junction tables (USER_SKILLS, USER_INTERESTS) instead of being stored as comma-separated lists.
- **2NF:** Every table has a single-column UUID primary key, so there are no partial dependencies on a composite key to remove.
- **3NF:** Non-key attributes depend only on the primary key. A team's readiness_score is stored once on TEAMS; a member's role is stored on TEAM_MEMBERS but is always traceable to (and consistent with) the proposed_role_id on the originating TEAM_INVITATIONS row, rather than being independently re-entered.

5.6 Key Design Decisions

- **Invitation as the only path to membership:** TEAM_MEMBERS.invitation_id is a required foreign key to an accepted TEAM_INVITATIONS row. There is no way to construct a valid membership record without first having gone through the invitation lifecycle, which enforces “the AI only suggests, it never adds” at the schema level, not just in application code.
- **MATCH_SUGGESTIONS is fully separate from TEAM_INVITATIONS:** keeping AI-generated candidate suggestions in their own table — rather than writing them directly as pending invitations — makes it impossible for a suggestion to be mistaken for, or silently become, a real invitation. A leader must take an explicit action (POST /teams/{id}/match/{id}/invite) to convert one into the other.
- **leader_id on TEAMS:** having a single, explicit leader field (rather than inferring leadership from “first member to join”) makes role-based authorization straightforward: only the user matching leader_id may send invitations or remove members.
- **Role is proposed at invitation time, not assigned after the fact:** proposed_role_id lives on TEAM_INVITATIONS, not as a separate “auto role assignment” step performed after members join. This matches the rule that every invitation — AI-suggested or not — already specifies the position being offered.
- **Reputation as an event log, not a counter:** REPUTATION_LOGS stores individual point-affecting events instead of a single mutable score field, preserving history and supporting tiered access privileges.
- **UUID primary keys:** chosen over auto-incrementing integers to avoid exposing sequential, guessable IDs through the FastAPI REST endpoints.
- **Catalog tables for SKILLS, INTERESTS, and ROLES:** kept as controlled reference lists rather than free text, so the AI matching engine can compare structured values when suggesting candidates and roles.

CHAPTER 6

UI /UX Design

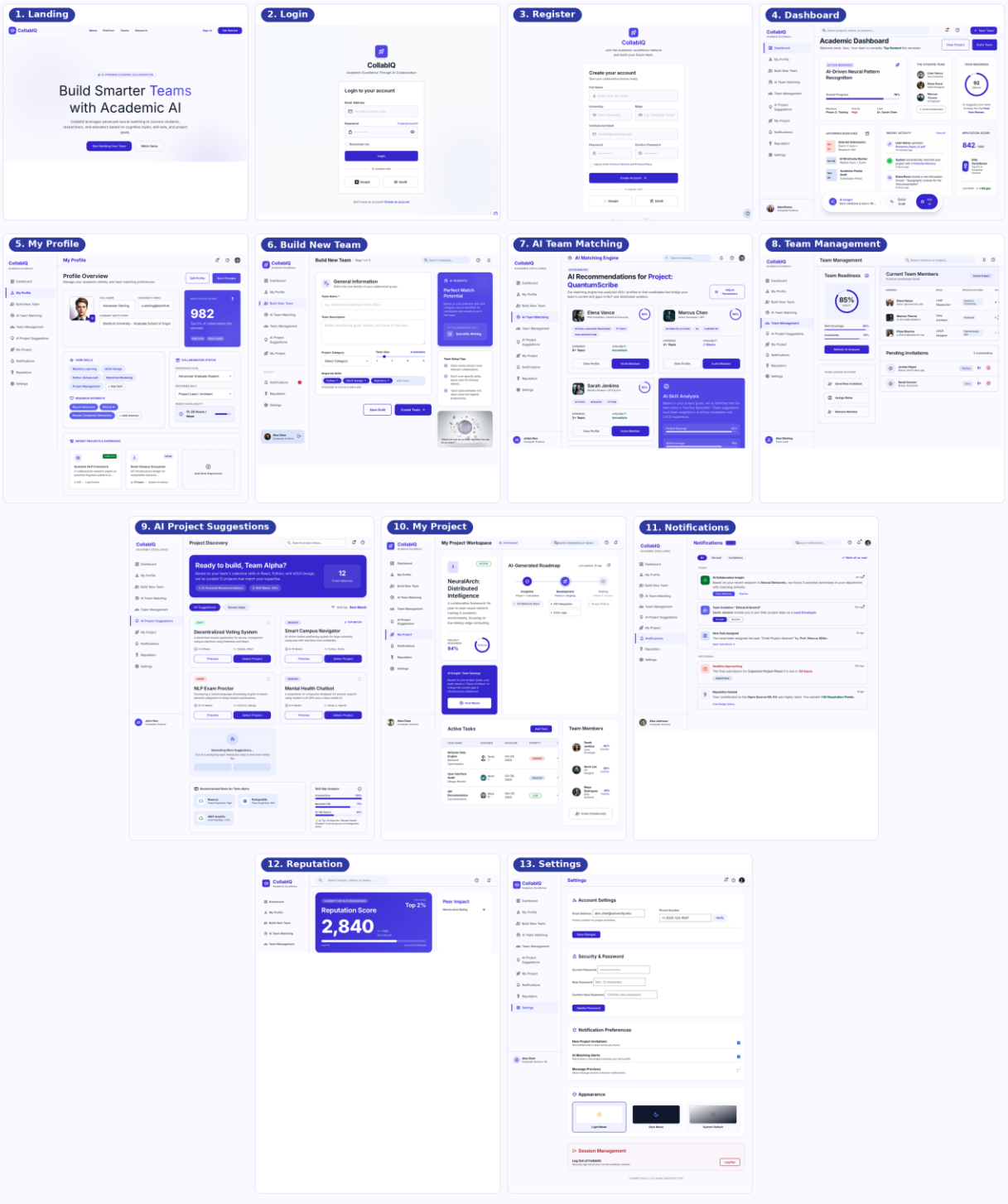
Member 6 — UI/ UX Design lead

6.1 Screen Inventory

Screen	Purpose
Landing	Marketing entry point; introduces CollabIQ and routes visitors to sign in or get started.
Login	Authenticates a returning student and issues a JWT access token (FR-1.2).
Register	Creates a new student account with name, email, and password (FR-1.1).
Dashboard	Central hub after login; surfaces team status, deadlines, activity, and quick actions.
My Profile	Profile, skills, and interests management required before team formation (FR-1.3–FR-1.6).
Build New Team	Team creation flow; the creating student becomes Team Leader (FR-2.1).
AI Team Matching	Displays AI-suggested candidate teammates with proposed roles and match scores (FR-2.7–FR-2.9).
Team Management	Leader view for sending/withdrawing invitations, managing members, and readiness score (FR-2.2–FR-2.6, FR-2.10–FR-2.11).
AI Project Suggestions	Ranked AI-generated project recommendations for the staffed team (FR-3.1–FR-3.4).
My Project	Active project workspace showing the AI-generated roadmap and task board (FR-3.5–FR-3.6, FR-4.1–FR-4.3).
Notifications	Invitation responses, task assignments, and reputation alerts.
Reputation	Displays the student's reputation score and contributing activity log (FR-4.4–FR-4.6).
Settings	Account, security, and notification preference management.

CollabIQ

AI-Powered Project Team Builder & Collaboration Platform



CollabIQ helps students build better teams, choose the right projects, and collaborate effectively with the power of AI.